

File Handling Assignment

File-based Caesar Cipher Encryption & Decryption

Level: Beginner to Intermediate

Topics Covered: File I/O, Caesar Cipher, String manipulation, Exception handling

Problem Statement:

You are tasked with building a simple cryptographic tool that reads a plain text file, encrypts its content using a **Caesar Cipher**, and then decrypts it back to the original content.

Detailed Requirements:

1. **Read File Content**
 - Read the contents of a file named `input.txt` from the local file system.
 - Assume the file contains plain text (no special formatting or binary data).
2. **Encrypt Using Caesar Cipher**
 - Ask the user for a **shift key (integer)**.
 - Apply **Caesar Cipher** to shift each alphabetic character by the given key.
 - Example: For a shift of 2, $A \rightarrow C$, $B \rightarrow D$, $Z \rightarrow B$
 - Non-alphabetic characters (like spaces, punctuation, digits) should be left unchanged.
 - Save the encrypted content to a new file called `encrypted.txt`.
3. **Decrypt Back**
 - Read from `encrypted.txt`, and apply the **reverse Caesar Cipher** using the same shift key.
 - Save the decrypted text to a file named `decrypted.txt`.
4. **Custom Exception**
 - Create a custom exception `InvalidShiftKeyException` that is thrown if the user enters a non-positive shift key (e.g., 0 or negative or more than 5).

Example Flow:

```
Input File: input.txt
Content: "Hello World!"
```

```
User Input: Shift Key = 3
```

```
Encrypted File: encrypted.txt
Content: "Khoor Zruog!"
```

```
Decrypted File: decrypted.txt
Content: "Hello World!"
```

Constraints:

- Uppercase and lowercase letters should maintain their case.
- Shift should wrap around the alphabet (e.g., z with shift 2 becomes b).

Deliverables:

- Java source files (.java) for reading, encryption, decryption, and exception handling.
- The text files used and created (input.txt, encrypted.txt, decrypted.txt).